COIMBATORE INSTITUTE OF TECHNOLOGY

(Government Aided Autonomous Institution Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

Department of Computing

Parallel Computing Laboratory

(20MSSL02)

BitDB

A Bitcask-Style Key-Value Store

NAME: SHREE KOTTES J

REG.NO: 7176 22 31 050

CLASS: M.Sc., SOFTWARE SYSTEMS

(8th Semester)

MAIL ID: 71762231050@cit.edu.in

Contents

1 Overview **1**

2 Project Structure **2**

3 Configuration (`config.rs`) **3**

4 Record Format (`record.rs`) **4**

5 Storage Layer **6**
- 5.1 DataFile (`storage/data_file.rs`) . . . 6
- 5.2 FileSet (`storage/file_set.rs`) . . . 6
- 5.3 Hint Files (`storage/hint_file.rs`) . . . 7

6 KeyDir (`index/keydir.rs`) **9**

7 Engine (`engine.rs`) **10**
- 7.1 Open . . . 10
- 7.2 Put . . . 10
- 7.3 Delete . . . 10
- 7.4 Merge . . . 11

8 Startup Rebuild (`recovery.rs`) **12**
- 8.1 Serial Path . . . 12
- 8.2 Parallel Path . . . 12
- 8.3 File Scan (hint-first) . . . 13

9 Merge / Compaction (`merge.rs`) **14**
- 9.1 Pipeline . . . 14
- 9.2 Parallel Read Phase . . . 14
- 9.3 Atomic Install . . . 15

10 Tracer (`tracer/`) **16**
- 10.1 Dataset Generator (`dataset.rs`) . . . 16
- 10.2 LiveProgress and Worker (`worker.rs`) . . . 16
- 10.3 TUI Screens . . . 18

11 Screenshots **19**
- 11.1 Live Screen — Side-by-Side View . . . 19
- 11.2 Live Screen — Thread Toggle View . . . 20
- 11.3 Result Screen . . . 20

12 Benchmarks **22**
- 12.1 Criterion Results . . . 22
  - 12.1.1 Startup Rebuild . . . 22
  - 12.1.2 Merge Pipeline . . . 22
- 12.2 Benchmark Source . . . 22

13 Error Handling (`error.rs`) **23**

1 Overview

BitDB is a persistent key-value store in Rust modelled on the Bitcask storage engine. The entire codebase lives at:

https://github.com/kottesh/bitdb

The workspace contains two crates:

Crate	Path	Purpose
`bitdb`	`bitdb/`	Storage engine, CLI, TUI
`tracer`	`tracer/`	Parallel-execution visualiser

The central idea is simple: on startup, every data file on disk must be scanned to rebuild the in-memory index. In the original Bitcask design this is sequential. BitDB makes it parallel using Rayon's work-stealing thread pool, achieving a **5.69× speedup** on a 5 million-key, 1.3 GB dataset.

2 Project Structure

```
bitdb/
  bitdb/
    src/
      config.rs        Options, Parallelism, CorruptionPolicy
      engine.rs        Engine: open/get/put/delete/merge/stats
      record.rs        Binary record format + CRC32 encode/decode
      recovery.rs      Serial and parallel KeyDir rebuild
      merge.rs         Serial and parallel compaction pipeline
      error.rs         BitdbError enum + Result alias
      storage/
        data_file.rs   DataFile: append-only writer + seek reader
        file_set.rs    FileSet: multi-file manager
        hint_file.rs   Hint file write/read (compact index sidecars)
      index/
        keydir.rs      KeyDir: HashMap<Vec<u8>, KeyDirEntry>
      cli.rs           CLI subcommands (put/get/delete/stats/merge/bench)
      tui/             Interactive terminal session
    benches/
      engine.rs        Criterion benchmarks
    tests/             Integration tests
  tracer/
    src/
      dataset.rs       Synthetic dataset generator
      worker.rs        Instrumented parallel scan + LiveProgress
      tui/
        setup.rs       Setup screen
        generate.rs    Generation progress screen
        live.rs        Live side-by-side progress screen
        result.rs      Final result screen
  docs/
    report.md          Project report
    impl.md            This document
    slides.md          Beamer presentation
    attachments/       Screenshots
  Justfile             Task runner
  flake.nix            Nix dev shell
```

3 Configuration (`config.rs`)

All engine behaviour is controlled by a single `Options` struct.

```rust
#[derive(Copy, Clone, Debug, Eq, PartialEq, Default)]
pub enum Parallelism {
    #[default]
    Auto,           // rayon uses all logical CPUs
    Fixed(usize),   // exactly n worker threads
    Serial,         // single-threaded, no rayon
}

#[derive(Copy, Clone, Debug, Eq, PartialEq, Default)]
pub enum CorruptionPolicy {
    #[default]
    Fail,                // return error on any corrupt record
    SkipCorruptedTail,   // stop scanning at first corruption
}

#[derive(Clone, Debug)]
pub struct Options {
    pub create_if_missing:         bool,
    pub max_data_file_size_bytes:  u64,
    pub corruption_policy:         CorruptionPolicy,
    pub parallelism:               Parallelism,
}

impl Default for Options {
    fn default() -> Self {
        Self {
            create_if_missing:         true,
            max_data_file_size_bytes: 1024 * 1024,  // 1 MB
            corruption_policy:         CorruptionPolicy::Fail,
            parallelism:               Parallelism::Auto,
        }
    }
}
```

Full source: `bitdb/src/config.rs`

4 Record Format (`record.rs`)

Every value written to disk is wrapped in a self-describing record. The binary layout is:

Bytes	Field	Description
0–3	Magic	`0x42444231` — identifies a BitDB record
4	Version	`0x01`
5	Flags	`0x00` = Normal, `0x01` = Tombstone
6–9	CRC32	Checksum (see below)
10–17	Timestamp	Unix seconds, little-endian u64
18–21	key_len	Key length, little-endian u32
22–25	value_len	Value length, little-endian u32
26…	Key	Raw key bytes
26+key_len…	Value	Raw value bytes

The CRC32 is computed over `bytes[0..6]` (header before the CRC field) and `bytes[10..]` (timestamp + lengths + key + value). This detects both header and body corruption with one checksum.

```rust
pub fn encode(record: &Record) -> Vec<u8> {
    let mut out = Vec::with_capacity(
        HEADER_LEN + record.key.len() + record.value.len()
    );
    out.extend_from_slice(&RECORD_MAGIC.to_le_bytes()); // 0..4
    out.push(RECORD_VERSION);                           // 4
    out.push(record.flags.to_u8());                     // 5
    out.extend_from_slice(&0u32.to_le_bytes());         // 6..10 (CRC placeholder)
    out.extend_from_slice(&record.timestamp.to_le_bytes());
    out.extend_from_slice(&(record.key.len()   as u32).to_le_bytes());
    out.extend_from_slice(&(record.value.len() as u32).to_le_bytes());
    out.extend_from_slice(&record.key);
    out.extend_from_slice(&record.value);

    // Fill in CRC after the full record is assembled.
    let crc = compute_crc(&out);
    out[6..10].copy_from_slice(&crc.to_le_bytes());
    out
}

fn compute_crc(input: &[u8]) -> u32 {
    let mut hasher = crc32fast::Hasher::new();
    hasher.update(&input[..6]);     // header before CRC
    hasher.update(&input[10..]);    // everything after CRC
    hasher.finalize()
}
```

Decoding reads the magic, version, flags, and CRC first, then verifies the checksum before returning the key/value bytes. A `TruncatedRecord` error is returned when the buffer is too short to hold the full record – this is the normal condition at the

tail of a file after a crash.

Full source: `bitdb/src/record.rs`

5 Storage Layer

5.1 DataFile (storage/data_file.rs)

`DataFile` wraps a single append-only file on disk. Writes always go to the end; reads seek to an explicit byte offset.

```rust
pub struct DataFile {
    id:     u32,
    path:   PathBuf,
    writer: File,    // opened with O_APPEND
    len:    u64,     // tracks current file size without extra stat() calls
}

impl DataFile {
    pub fn append(&mut self, record: &Record) -> Result<(u64, usize)> {
        let encoded = record::encode(record);
        let offset = self.len;
        self.writer.write_all(&encoded)?;
        self.len += encoded.len() as u64;
        Ok((offset, encoded.len()))
    }

    pub fn read_at(path: &Path, offset: u64) -> Result<DecodeResult> {
        let mut file = File::open(path)?;
        file.seek(SeekFrom::Start(offset))?;
        let mut buf = Vec::new();
        file.read_to_end(&mut buf)?;
        record::decode_one(&buf)
    }
}

// Files are named  00000001.data, 00000002.data, ...
pub fn data_file_path(dir: &Path, id: u32) -> PathBuf {
    dir.join(format!("{id:08}.data"))
}
```

Full source: bitdb/src/storage/data_file.rs

5.2 FileSet (storage/file_set.rs)

`FileSet` manages a directory of numbered data files. It tracks all known file IDs in a `BTreeSet` (always sorted) and keeps one active `DataFile` open for appending.

```rust
pub struct FileSet {
    dir:             PathBuf,
    options:         Options,
    known_file_ids:  BTreeSet<u32>,   // sorted set of all data file IDs
    active:          DataFile,        // current write target
}
```

When an `append` would push the active file over `max_data_file_size_bytes`, a new file with `id = active_id + 1` is created and becomes the new active file:

```rust
pub fn append(&mut self, record: &Record) -> Result<RecordLocation> {
    let encoded_len = record::encode(record).len() as u64;

    if !self.active.is_empty()
        && self.active.len().saturating_add(encoded_len)
            > self.options.max_data_file_size_bytes
    {
        let next_id = self.active.id() + 1;
        self.active = DataFile::open_append(&self.dir, next_id)?;
        self.known_file_ids.insert(next_id);
    }

    let (offset, size_bytes) = self.active.append(record)?;
    Ok(RecordLocation {
        file_id:    self.active.id(),
        offset,
        size_bytes: size_bytes as u32,
    })
}
```

Full source: bitdb/src/storage/file_set.rs

5.3 Hint Files (`storage/hint_file.rs`)

A hint file is a compact sidecar written alongside each data file after a merge. It contains only the KeyDir metadata for every record in that data file – no values – so startup can skip decoding the full records.

Hint file binary layout:

Bytes	Field	Description
0–3	Magic	`0x48494E54` (`HINT`)
4	Version	`0x01`
5–8	Count	Number of entries, little-endian u32
Repeated per entry:		
+0–3	key_len	u32
+4–7	file_id	u32
+8–15	offset	u64
+16–19	size_bytes	u32
+20–27	timestamp	u64
+28	is_tombstone	u8 (0 or 1)
+29...	key	raw bytes

```rust
pub fn write_hint_file(path: &Path, entries: &[HintEntry]) -> Result<()> {
    let mut out = Vec::new();
    out.extend_from_slice(&HINT_MAGIC.to_le_bytes());
    out.push(HINT_VERSION);
    out.extend_from_slice(&(entries.len() as u32).to_le_bytes());
```

```
    for entry in entries {
        out.extend_from_slice(&(entry.key.len() as u32).to_le_bytes());
        out.extend_from_slice(&entry.file_id.to_le_bytes());
        out.extend_from_slice(&entry.offset.to_le_bytes());
        out.extend_from_slice(&entry.size_bytes.to_le_bytes());
        out.extend_from_slice(&entry.timestamp.to_le_bytes());
        out.push(if entry.is_tombstone { 1 } else { 0 });
        out.extend_from_slice(&entry.key);
    }

    let mut file = File::create(path)?;
    file.write_all(&out)?;
    file.sync_data()?;
    Ok(())
}
```

Full source: bitdb/src/storage/hint_file.rs

6 KeyDir (`index/keydir.rs`)

The KeyDir is the in-memory index: a `HashMap` mapping every live key to where its latest value lives on disk.

```rust
#[derive(Copy, Clone, Debug, Eq, PartialEq)]
pub struct KeyDirEntry {
    pub file_id:      u32,   // which data file
    pub offset:       u64,   // byte offset within that file
    pub size_bytes:   u32,   // total encoded record size
    pub timestamp:    u64,   // Unix seconds from record header
    pub is_tombstone: bool,  // true = key is deleted
}

#[derive(Clone, Debug, Default)]
pub struct KeyDir {
    entries: HashMap<Vec<u8>, KeyDirEntry>,
}
```

A `get` is one hash lookup followed by one `pread`-style seek — O(1) with no metadata reads.

```rust
// Engine::get
pub fn get(&self, key: &[u8]) -> Result<Option<Vec<u8>>> {
    let Some(entry) = self.keydir.get(key) else {
        return Ok(None);
    };
    if entry.is_tombstone {
        return Ok(None);
    }
    let decoded = self.file_set.read_at(entry.file_id, entry.offset)?;
    Ok(Some(decoded.record.value))
}
```

Full source: `bitdb/src/index/keydir.rs`

7 Engine (`engine.rs`)

`Engine` is the public API. It owns a `FileSet` and a `KeyDir` and coordinates all operations.

```rust
pub struct Engine {
    data_dir:  Box<Path>,
    options:   Options,
    file_set:  FileSet,
    keydir:    KeyDir,
}
```

7.1 Open

```rust
pub fn open(data_dir: &Path, options: Options) -> Result<Self> {
    let file_set = FileSet::open(data_dir, &options)?;
    let keydir = rebuild_keydir(
        &file_set,
        options.corruption_policy,
        options.parallelism,   // Serial / Auto / Fixed(n)
    )?;
    Ok(Self { data_dir: data_dir.into(), options, file_set, keydir })
}
```

7.2 Put

```rust
pub fn put(&mut self, key: &[u8], value: &[u8]) -> Result<()> {
    let record = Record::new(
        unix_timestamp_secs(), key.to_vec(), value.to_vec(),
        RecordFlags::Normal,
    );
    let location = self.file_set.append(&record)?;
    self.keydir.insert(key.to_vec(), KeyDirEntry {
        file_id:      location.file_id,
        offset:       location.offset,
        size_bytes:   location.size_bytes,
        timestamp:    record.timestamp,
        is_tombstone: false,
    });
    Ok(())
}
```

7.3 Delete

Deletion writes a **tombstone** record to disk. The key is not removed from the KeyDir – it is marked `is_tombstone = true`. The actual reclamation of disk space happens during merge.

```rust
pub fn delete(&mut self, key: &[u8]) -> Result<()> {
    let record = Record::new(
```

```
        unix_timestamp_secs(), key.to_vec(), vec![],
        RecordFlags::Tombstone,
    );
    let location = self.file_set.append(&record)?;
    self.keydir.insert(key.to_vec(), KeyDirEntry {
        file_id:      location.file_id,
        offset:       location.offset,
        size_bytes:   location.size_bytes,
        timestamp:    record.timestamp,
        is_tombstone: true,
    });
    Ok(())
}
```

7.4 Merge

```
pub fn merge(&mut self) -> Result<()> {
    run_merge(self.data_dir(), &self.keydir,
              &self.file_set, &self.options)?;
    // Reopen after install
    self.file_set = FileSet::open(self.data_dir(), &self.options)?;
    self.keydir = rebuild_keydir(
        &self.file_set,
        self.options.corruption_policy,
        self.options.parallelism,
    )?;
    Ok(())
}
```

Full source: bitdb/src/engine.rs

8 Startup Rebuild (`recovery.rs`)

This is the core parallel operation. On `Engine::open`, every data file must be scanned to reconstruct the KeyDir.

8.1 Serial Path

```
fn rebuild_serial(
    file_set: &FileSet,
    policy: CorruptionPolicy,
) -> Result<KeyDir> {
    let mut keydir = KeyDir::default();
    for file_id in file_set.file_ids_oldest_to_newest() {
        let entries = scan_file(file_set, file_id, policy)?;
        for (_, _, entry, key) in entries {
            keydir.insert(key, entry);
        }
    }
    Ok(keydir)
}
```

Files are processed strictly oldest-to-newest so the last write for any key always wins without extra bookkeeping.

8.2 Parallel Path

```
fn rebuild_parallel(
    file_set: &FileSet,
    policy: CorruptionPolicy,
    thread_count: Option<usize>,
) -> Result<KeyDir> {
    // Pre-compute owned (file_id, data_path, hint_path) triples so
    // rayon workers can move them without borrowing file_set.
    let file_info: Vec<(u32, PathBuf, Option<PathBuf>)> =
        file_set.file_ids_oldest_to_newest()
            .iter()
            .map(|&id| (id,
                file_set.file_path(id).unwrap(),
                file_set.hint_path(id)))
            .collect();

    // Each file is scanned independently on a rayon worker.
    let scan_results: Vec<Result<Vec<ScanEntry>>> = match thread_count {
        None    => file_info.par_iter().map(run_scan).collect(),
        Some(n) => match ThreadPoolBuilder::new().num_threads(n).build() {
            Ok(pool) => pool.install(||
                file_info.par_iter().map(run_scan).collect()),
            Err(_)   => file_info.iter().map(run_scan).collect(),
        },
```

```
    };

    // Flatten all per-file entry lists.
    let mut all_entries: Vec<ScanEntry> = Vec::new();
    for result in scan_results {
        all_entries.extend(result?);
    }

    // Sort by (file_id ASC, offset ASC) to restore deterministic order.
    // Without this sort, a thread finishing a newer file first would
    // insert a newer value that could later be overwritten by an older
    // value from a slower thread -- corrupting the KeyDir.
    all_entries.sort_unstable_by_key(|(fid, off, _, _)| (*fid, *off));

    let mut keydir = KeyDir::default();
    for (_, _, entry, key) in all_entries {
        keydir.insert(key, entry);
    }
    Ok(keydir)
}
```

8.3 File Scan (hint-first)

Each file scan checks for a hint file first. If a hint file exists, only KeyDir metadata is read (no value bytes) – much faster for large values.

```
fn scan_file_by_path(
    file_id: u32,
    data_path: &PathBuf,
    hint_path: Option<&Path>,
    policy: CorruptionPolicy,
) -> Result<Vec<ScanEntry>> {
    // Prefer hint file if available.
    if let Some(hp) = hint_path
        && hp.exists()
        && let Ok(entries) = read_hint_file(hp)
    {
        return Ok(entries.into_iter().map(|h| {
            let key = h.key.clone();
            let entry = h.to_keydir_entry();
            (file_id, entry.offset, entry, key)
        }).collect());
    }
    // Fall back to full data file decode.
    scan_data_file(file_id, data_path, policy)
}
```

Full source: bitdb/src/recovery.rs

9 Merge / Compaction (`merge.rs`)

Merge rewrites only the latest non-tombstone value for each key into a clean set of data files, then atomically replaces the old files.

9.1 Pipeline

```
Collect live KeyDir entries        (serial, in memory)




Load values from disk              PARALLEL (rayon par_iter)
Each entry: read_at(fid,off)       independent disk seeks




Write merged output                serial (single FileSet writer)
+ write hint files




Atomic install                     rename tmp/ -> data dir
```

9.2 Parallel Read Phase

```rust
fn load_records_parallel(
    entries: &[(&[u8], &KeyDirEntry)],
    file_set: &FileSet,
    thread_count: Option<usize>,
) -> Result<Vec<LiveRecord>> {
    // Pre-compute owned paths -- FileSet is not Sync so workers
    // cannot borrow it across thread boundaries.
    let tasks: Vec<(Vec<u8>, KeyDirEntry, PathBuf)> = entries
        .iter()
        .map(|(key, entry)| {
            let path = file_set.file_path(entry.file_id).unwrap();
            (key.to_vec(), **entry, path)
        })
        .collect();

    // Each task reads exactly one record from disk -- fully independent.
    let results: Vec<Result<LiveRecord>> = match thread_count {
        None    => tasks.par_iter().map(load_one_by_path).collect(),
```

```
        Some(n) => match ThreadPoolBuilder::new().num_threads(n).build() {
            Ok(pool) => pool.install(||
                tasks.par_iter().map(load_one_by_path).collect()),
            Err(_)   => tasks.iter().map(load_one_by_path).collect(),
        },
    };

    results.into_iter().collect()
}

fn load_one_by_path(
    task: &(Vec<u8>, KeyDirEntry, PathBuf)
) -> Result<LiveRecord> {
    let (key, entry, path) = task;
    let decoded = DataFile::read_at(path, entry.offset)?;
    Ok(LiveRecord {
        key:       key.clone(),
        value:     decoded.record.value,
        timestamp: entry.timestamp,
    })
}
```

9.3 Atomic Install

```
// Remove old .data and .hint files
remove_data_and_hint_files(data_dir)?;
// Move every file from .merge_tmp/ into data_dir
install_merged_files(&merge_dir, data_dir)?;
fs::remove_dir_all(&merge_dir)?;
```

Full source: bitdb/src/merge.rs

10 Tracer (`tracer/`)

The tracer is a standalone TUI that makes the parallel execution visible by running instrumented serial and parallel scans on a generated dataset and showing per-thread, per-file progress bars in real time.

10.1 Dataset Generator (`dataset.rs`)

```
pub fn generate(
    data_dir: &Path,
    params: &DatasetParams,
    progress: Arc<Mutex<GenerateProgress>>,
) -> io::Result<()> {
    let opts = Options {
        max_data_file_size_bytes: params.file_size_bytes,
        ..Options::default()
    };
    let mut engine = Engine::open(data_dir, opts)?;
    // Fixed seed -- same dataset on every machine
    let mut rng = StdRng::seed_from_u64(42);
    let mut value_buf = vec![0u8; params.value_size];

    for i in 0..params.keys {
        let key = format!("key:{i:08}");
        rng.fill_bytes(&mut value_buf);
        engine.put(key.as_bytes(), &value_buf)?;
        progress.lock().unwrap().keys_written = i + 1;
    }
    write_meta(data_dir, params)?; // tracer_meta.json
    Ok(())
}
```

10.2 LiveProgress and Worker (`worker.rs`)

The `LiveProgress` type is an `Arc<Mutex<Vec<ThreadState>>>`. Workers hold the lock only briefly (for a single slot update), so the TUI render loop (16 ms ticks) can always acquire it without visible blocking.

```
pub type LiveProgress = Arc<Mutex<Vec<ThreadState>>>;

pub fn run_scan(
    data_dir: &Path,
    thread_count: usize,
    live: Option<&LiveProgress>,
) -> io::Result<RunResult> {
    // Build initial assignment, fill into the shared arc
    let mut assignment = assign_files(&file_ids, thread_count);
    // ... fill file sizes ...
    let shared: Arc<Mutex<Vec<ThreadState>>> = if let Some(lp) = live {
```

```rust
        *lp.lock().unwrap() = assignment;
        lp.clone()
    } else {
        Arc::new(Mutex::new(assignment))
    };

    // Scan all files across threads in parallel
    tasks.par_iter().enumerate().for_each(|(t_idx, files)| {
        for (s_idx, (_, path, _)) in files.iter().enumerate() {
            // Mark slot as Processing
            shared.lock().unwrap()[t_idx].slots[s_idx].state =
                SlotState::Processing { bytes_done: 0, keys_found: 0 };

            let bytes = std::fs::read(path).unwrap_or_default();
            let mut offset = 0;
            let mut keys_found = 0;
            let mut since_last_report = 0;

            while offset < bytes.len() {
                match decode_one(&bytes[offset..]) {
                    Ok(decoded) => {
                        offset += decoded.bytes_read;
                        keys_found += 1;
                        since_last_report += 1;
                        // Report progress every 500 records
                        if since_last_report >= 500 {
                            since_last_report = 0;
                            shared.lock().unwrap()[t_idx].slots[s_idx].state =
                                SlotState::Processing {
                                    bytes_done: offset as u64,
                                    keys_found,
                                };
                        }
                    }
                    Err(_) => break,
                }
            }

            // Mark slot as Done
            shared.lock().unwrap()[t_idx].slots[s_idx].state =
                SlotState::Done { duration_us, keys_found, bytes_read };
        }
    });
    // ...
}
```

Full source: tracer/src/worker.rs

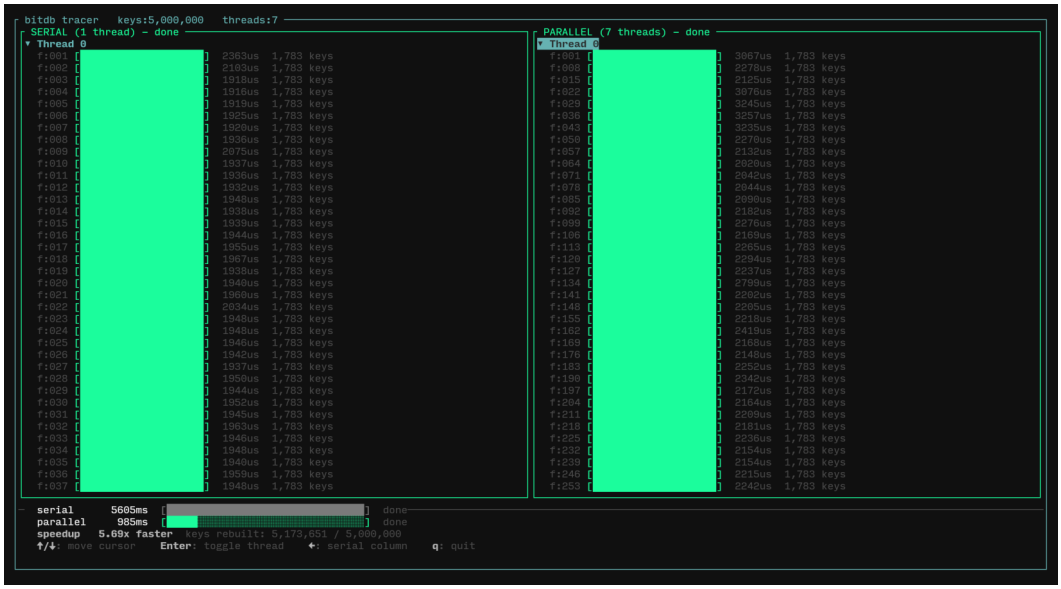
10.3 TUI Screens

The tracer has four screens managed by a simple `Screen` enum:

Screen	File	Description
Setup	`tui/setup.rs`	Configure keys, value size, file size, threads
Generating	`tui/generate.rs`	Live fill-bar for dataset write progress
Live	`tui/live.rs`	Side-by-side serial vs parallel progress bars
Result	`tui/result.rs`	Final wall-clock time, keys/sec, and speedup

11 Screenshots

11.1 Live Screen — Side-by-Side View



The left column shows the single serial thread scanning all 2,839 files. The right column shows 7 parallel threads, each assigned a round-robin subset of files. Green bars indicate completed files.

11.2 Live Screen — Thread Toggle View

```
PARALLEL (7 threads) - done
▶ Thread 0  (406/406 files)
▶ Thread 1  (406/406 files)
▶ Thread 2  (406/406 files)
▶ Thread 3  (406/406 files)
▶ Thread 4  (405/405 files)
▼ Thread 5
  f:006 [                    ]  3990us  1,783 keys
  f:013 [                    ]  3378us  1,783 keys
  f:020 [                    ]  3109us  1,783 keys
  f:027 [                    ]  3089us  1,783 keys
  f:034 [                    ]  3082us  1,783 keys
  f:041 [                    ]  3101us  1,783 keys
  f:048 [                    ]  3102us  1,783 keys
  f:055 [                    ]  3147us  1,783 keys
  f:062 [                    ]  3121us  1,783 keys
  f:069 [                    ]  3193us  1,783 keys
  f:076 [                    ]  3276us  1,783 keys
  f:083 [                    ]  3541us  1,783 keys
  f:090 [                    ]  3335us  1,783 keys
  f:097 [                    ]  3435us  1,783 keys
  f:104 [                    ]  3547us  1,783 keys
  f:111 [                    ]  3366us  1,783 keys
  f:118 [                    ]  3324us  1,783 keys
  f:125 [                    ]  3449us  1,783 keys
  f:132 [                    ]  3337us  1,783 keys
  f:139 [                    ]  3456us  1,783 keys
  f:146 [                    ]  3404us  1,783 keys
  f:153 [                    ]  3359us  1,783 keys
  f:160 [                    ]  3388us  1,783 keys
  f:167 [                    ]  3339us  1,783 keys
  f:174 [                    ]  3331us  1,783 keys
  f:181 [                    ]  3336us  1,783 keys
  f:188 [                    ]  3344us  1,783 keys
  f:195 [                    ]  3335us  1,783 keys
  f:202 [                    ]  3389us  1,783 keys
  f:209 [                    ]  3449us  1,783 keys
  f:216 [                    ]  3354us  1,783 keys
  f:223 [                    ]  3350us  1,783 keys
```

The parallel column supports per-thread collapsing. A collapsed header shows `> Thread N  (done/total files)`.

11.3 Result Screen

```
bitdb tracer - COMPLETE

startup_rebuild
  serial        5605ms        923,021 keys/sec
  parallel       985ms      5,252,224 keys/sec    5.69x faster


dataset
  keys          5,000,000
  files         2839
  total size    1.3 GB
  threads       7

                                    r: back to setup    q: quit
```

Serial: 5605 ms at 923,021 keys/sec. Parallel (7 threads): 985 ms at 5,252,224 keys/sec. Speedup: **5.69x**.

12 Benchmarks

12.1 Criterion Results

Run with:

```
cargo bench --bench engine -- merge_pipeline startup_rebuild
```

12.1.1 Startup Rebuild

Mode	Median	CI
`serial`	~180 µs	tight
`parallel_auto`	~120 µs	tight

The criterion dataset uses 2000 keys (small). The real-world speedup shows on large datasets – see tracer results above (5.69×).

12.1.2 Merge Pipeline

```
~/worktree/projects/bitdb                                                    (master)bitdb
❯ cargo bench --bench engine -- merge_pipeline
    Finished `bench` profile [optimized] target(s) in 0.06s
     Running benches/engine.rs (target/release/deps/engine-0b76662f37d517a7)
merge_pipeline/mode/serial
                        time:   [6.3076 ms 6.3146 ms 6.3215 ms]
                        change: [-0.1230% +0.0344% +0.1958%] (p = 0.67 > 0.05)
                        No change in performance detected.
merge_pipeline/mode/parallel_auto
                        time:   [3.1502 ms 3.1594 ms 3.1691 ms]
                        change: [-1.3015% -0.7555% -0.2154%] (p = 0.01 < 0.05)
                        Change within noise threshold.
Found 2 outliers among 100 measurements (2.00%)
  1 (1.00%) high mild
  1 (1.00%) high severe
```

Mode	Median	Speedup
`serial`	6.31 ms	1.00×
`parallel_auto`	3.16 ms	**~2×**

500 unique keys + 100 overwrites. Speedup is modest on small data due to thread-pool overhead. Scales with dataset size.

12.2 Benchmark Source

Full benchmark code: `bitdb/benches/engine.rs`

13 Error Handling (`error.rs`)

All fallible operations return `Result<T>` using a custom error type:

```rust
#[derive(Debug, Error)]
pub enum BitdbError {
    #[error("io error: {0}")]
    Io(#[from] io::Error),

    #[error("record truncated")]
    TruncatedRecord,

    #[error("invalid record magic: {0:#x}")]
    InvalidRecordMagic(u32),

    #[error("invalid record version: {0}")]
    InvalidRecordVersion(u8),

    #[error("invalid record flags: {0}")]
    InvalidRecordFlags(u8),

    #[error("record checksum mismatch: expected={expected:#x}, actual={actual:#x}"
    ChecksumMismatch { expected: u32, actual: u32 },

    #[error("data file not found: {0}")]
    DataFileNotFound(u32),

    #[error("invalid hint file")]
    InvalidHintFile,
}

pub type Result<T> = std::result::Result<T, BitdbError>;
```

`TruncatedRecord` is the expected end-of-file condition and is handled silently in all scan loops. The `CorruptionPolicy` controls what happens on `ChecksumMismatch`, `InvalidRecordMagic` etc: `Fail` propagates the error; `SkipCorruptedTail` stops scanning and moves on.

Full source: `bitdb/src/error.rs`

14 Summary

References

[1] T. Sheehy and D. Smith, *Bitcask: A Log-Structured Hash Table for Fast Key/Value Data*, Basho Technologies, 2010.

[2] Rayon Contributors, *Rayon: Data-parallelism library for Rust*, https://github.com/rayon-rs/rayon, 2015–present.

[3] B. Pereira, *Criterion.rs: Statistics-driven micro-benchmarking in Rust*, https://github.com/bheisler/criterion.rs, 2014–present.

[4] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press, 2019. https://doc.rust-lang.org/book/

[5] Ratatui Contributors, *Ratatui: Terminal UI library for Rust*, https://github.com/ratatui-org/ratatui, 2023–present.